

NutShell Cache 验证 报告

——基于 UCAgent + Claude Code + WorkBuddy 框架的功能验证与覆盖率闭环

Zhaoyue Zhang

2026 年 6 月

v1.0

目录

1	引言	4
1.1	项目背景	4
1.2	赛事要求	4
1.3	工具链	5
1.4	文档结构	5
2	验证平台	6
2.1	平台概述	6
2.2	平台组件	6
2.2.1	DUT (Design Under Test)	6
2.2.2	CacheWrapper (缓存封装器)	6
2.2.3	Reference Model (参考模型)	6
2.2.4	Scoreboard (计分板)	7
2.2.5	功能覆盖率模型	7
2.3	UCAgent 工作流	7
3	验证点	8
3.1	验证点分类	8
3.2	验证点 1: 基本功能	8
3.3	验证点 2: MMIO 访问	8
3.4	验证点 3: Cache 命中	8
3.5	验证点 4: Cache 缺失	8
3.6	验证点汇总	9
4	仿真环境	10
4.1	硬件配置	10
4.2	软件环境	10
4.3	安装与验证	11
4.4	回归测试	11
5	测试用例	12
5.1	Smoke Test (基本连通性测试)	12
5.2	Directed Tests (定向测试)	12
5.3	Random Tests (约束随机测试)	13
5.4	Bug 注入测试	14

6	覆盖报告	15
6.1	覆盖率总览	15
6.2	行覆盖率豁免分析	15
6.3	分支覆盖率豁免	15
6.4	翻转覆盖率豁免	16
6.5	Toffee 分支覆盖率报告差异	16
7	总结	18
7.1	工作亮点	18
7.2	不足与改进	18
7.3	未来展望	18
8	结论	19

1 引言

1.1 项目背景

本报告记录了 NutShell RISC-V 处理器 Cache 模块的完整功能验证过程。验证工作采用 UCAgent 框架辅助——通过 Claude Code 连接 UCAgent MCP Server 执行 AI 辅助验证，WorkBuddy 负责文档整理与报告生成，Makefile 统一管理 workflow 启动——结合 Picker (RTL→Python 导出)、Verilator 仿真器、pytest 测试框架及 Toffee 验证方法学，完成了从环境搭建到覆盖率闭环的全流程验证。

NutShell 是由中国科学院计算技术研究所开发的一款基于 RISC-V RV64 指令集的开源顺序处理器。其 Cache 子系统包括 L1 指令 Cache (I-Cache)、L1 数据 Cache (D-Cache) 和统一的 L2 Cache。本次验证聚焦于 L1 Cache 模块，具体实现为 Picker example/Cache 中的 4 路组相联 Cache 设计。

1.2 赛事要求

本验证项目基于 CCF Track1 UCAgent 竞赛框架。评分体系包括验证实操深度 (60 分)、报告与协同分析 (40 分)，并设置了 AI 协同记录和可复现性要求。

验证工作分为五大阶段：

阶段 1： RTL 理解与最小闭环 (Smoke Test)；

阶段 2： Toffee 风格验证环境构建 (Generator/Driver/Monitor/Scoreboard 拆分)；

阶段 3： CRV 与覆盖率闭环 (目标 90%+)；

阶段 4： 故障注入与检出能力证明 (至少若干 Bug 检出案例)；

阶段 5： 报告、复现与提交整理。

1.3 工具链

表 1: 验证工具链

工具/组件	版本	用途
UCAgent	0.9.2+	AI 辅助验证编排框架
Claude Code	0.131.0+	连接 UCAgent MCP Server，AI 辅助验证执行
WorkBuddy	latest	文档整理与报告生成
Makefile	—	工作流启动管理（回归/覆盖率/Bug 注入）
Picker	0.1.0-master	RTL→Python DUT 导出
Verilator	5.046	RTL 仿真引擎
pytest	8.1.1	Python 测试框架
pytoffee / toffee-test	latest	Toffee 验证方法学
Java (Zulu JRE)	17.0.19	NutShell Chisel 编译
Mill	0.11.7	Chisel 构建工具

1.4 文档结构

本文档共八章。第一章为引言，介绍项目背景和工具链。第二章描述验证平台的整体架构与组件设计。第三章列出所有验证点及其覆盖范围。第四章介绍仿真环境配置。第五章详述测试用例设计。第六章给出覆盖率分析报告。第七章总结工作亮点与不足。第八章为结论。

2 验证平台

2.1 平台概述

验证平台基于 Toffee 验证方法学构建，采用 Python + pytest 作为测试编排层。Picker 工具将 RTL 设计 (Cache.v) 导出为 Python 可调用的 DUT 对象 (DUTCACHE)，Verilator 作为底层仿真引擎执行周期精确仿真。

整体验证流程如下：

1. 使用 Picker 将 NutShell Cache RTL 导出为 Python DUT 类；
2. 通过 Toffee 风格的 Env/Generator/Driver/Monitor/Scoreboard 组件构建结构化验证环境；
3. 编写定向测试、随机测试和 Bug 注入测试用例；
4. 通过 pytest 执行测试，收集功能覆盖率和 RTL 代码覆盖率；
5. 使用 UCAGENT 编排验证阶段，记录 Stage Journal 证据。

2.2 平台组件

2.2.1 DUT (Design Under Test)

DUT 选定为 Picker example/Cache 目录下的 Cache.v (而非完整 NutShell Chisel 生成的 RTL)。该模块是一个 4 路组相联 Cache，包含以下子模块：CacheStage1 (请求解析)、CacheStage2 (SRAM 访问与 Tag 比较)、CacheStage3 (响应收集与替换控制)、Arbiter (多路仲裁)、SRAMTemplate (SRAM 行为模型)。

Cache 配置参数：L1 Cache 大小 32KB，Cache Line 大小 64B，采用 4 路组相联映射，替换策略为随机替换 (LFSR 伪随机)。支持 SimpleBus 协议 (Read/Write/ReadBurst/WriteBurst)，通过 MMIO 接口 (地址空间 0x30000000~0x7ffffff) 访问外设。

2.2.2 CacheWrapper (缓存封装器)

CacheWrapper 负责封装 DUT 的接口访问，提供统一的事务级操作接口。它维护请求队列 (req_queue) 和响应队列 (resp_queue)，处理 DUT 的 valid-ready 握手协议，确保测试激励正确驱动到 DUT 引脚。

2.2.3 Reference Model (参考模型)

参考模型 (Ref Cache) 实现 Cache 的功能等价行为，包括：SimpleMem (模拟不使用 Cache 时的内存行为)、Clean/Dirty Table (C/D Table，跟踪 Cache Line 状态)。参考模型在处理读写请求时模拟 Cache 的命中/缺失/替换/写回行为，为 Scoreboard 提供期望值。

2.2.4 Scoreboard（计分板）

Scoreboard 基于 Toffee 框架实现，采用 3 级结构（11 个检查方法）。核心检查包括：请求/响应队列跟踪、内存请求期望验证、读响应数据比对、写掩码验证、多节拍 Refill 数据完整性检查、脏写回序列验证、替换策略检查。Scoreboard 延迟比对 DUT 输出与参考模型的期望输出，不匹配时立即报错。Bug 注入后 Scoreboard 正确检出全部 6 个 Bug。

2.2.5 功能覆盖率模型

功能覆盖率模型基于 Toffee 框架的 fc_cover 机制实现，包含 12 个覆盖组（Coverage Groups）、31 个覆盖点（Coverage Points）、37 个覆盖仓（Coverage Bins），覆盖以下维度：命令类型（Read/Write/Probe）、命中/缺失、写掩码类型、字偏移、MMIO 访问、Flush 操作、Coherence 探针、Clean/Dirty 替换、Refill 路径。当前所有覆盖组均达到 100% 覆盖。

2.3 UCAgent workflow

验证过程通过 UCAgent 框架编排为 18 个 Stage（Stage 0–17），每个 Stage 包含 inspect→implement→verify→document 四步流程。关键 Stage 包括：

表 2: UCAgent Stage 概览

Stage	描述
Stage 0	源文件清单与 DUT 边界确定
Stage 1–2	接口映射与测试点分析
Stage 3	Smoke Test 最小闭环
Stage 4–7	结构化验证环境构建与定向测试
Stage 8–10	CRV 随机测试与覆盖率引导
Stage 11–12	行覆盖率闭环（100% → 100%）
Stage 13	分支覆盖率闭环（100%）
Stage 14–16	翻转覆盖率提升（86.7%→88.4%）
Stage 17	最终报告与提交检查

3 验证点

3.1 验证点分类

根据 Cache 的功能特性，验证点分为以下类别：基本功能（Cache Read/Write Hit）、MMIO 访问（Memory-Mapped I/O 读写与反压）、Cache 命中（读写命中验证与时延检查）、Cache 缺失（缺失处理、脏替换、Clean 替换）、Flush 行为、Coherence 探针。

3.2 验证点 1：基本功能

验证 Cache 对常规读写请求的正确响应。包括：Read Hit（命中时返回正确数据）、Write Hit（写入数据并正确更新 Cache Line）、Read-After-Write Hit（写后读一致性）、Write Mask（部分字节写入不影响同 Cache Line 其他字节）、Word Offset（同一 Cache Line 不同偏移的独立读写）。

对应测试用例：Smoke Test（SMK-001~007）、定向测试（DIR-001~013）。

3.3 验证点 2：MMIO 访问

验证 Cache 对 MMIO 地址空间的正确处理。MMIO 地址范围为 0x30000000~0x7ffffff, Cache 应对 MMIO 请求执行旁路（Bypass），不进行缓存。验证内容包括：MMIO 读写正确性（非 Burst）、MMIO Backpressure（反压时 Ready 信号拉低）、MMIO Burst 拒绝（MMIO 不应产生 Burst 传输）。

对应测试用例：MMIO Test（DIR-004）。

3.4 验证点 3：Cache 命中

验证 Cache 命中路径的正确性和性能。包括：命中时数据在 3 个周期内返回（时延约束）、命中时不产生不必要的 Memory 请求、LRU/随机替换策略不影响命中行为。

对应测试用例：Cache Hit Test、Random Test（CRV-001~005）。

3.5 验证点 4：Cache 缺失

验证 Cache 缺失处理路径的完整性和正确性。包括：

- Read Miss → Refill：读缺失触发内存 Refill，多节拍数据完整性检查；
- Write Miss → Refill：写缺失先 Refill 再写入；
- Dirty Eviction → Writeback → Refill：脏替换先写回脏数据再 Refill；
- Clean Eviction：洁净替换不产生写回；

- Invalid Way Priority：无效路优先替换。

对应测试用例：Cache Miss Test、定向测试（DIR-005~012）。

3.6 验证点汇总

表 3: 验证点汇总

编号	特性	覆盖范围	关联测试	状态
1	基本功能	Read/Write Hit, RAW, Mask, Offset	SMK, DIR-001~003,013	
2.1	MMIO 读写	MMIO R/W 正确性	DIR-004	
2.2	MMIO 反压	MMIO Ready 拉低	DIR-004	
2.3	MMIO Burst	MMIO 不产生 Burst	DIR-004	
3.1	Cache 命中时延	≤3 周期, 无多余 Mem 请求	CRV-001~005	
4.1	Read Miss+Refill	8 节拍数据完整性	DIR-005~006	
4.2	Dirty Eviction+WB	脏替换 → 写回 →Refill	DIR-007,012	
4.3	Clean Eviction	无写回	DIR-011	
4.4	Invalid Way	无效路优先替换	DIR-003	
5	Flush 行为	idle/in-flight flush, io_empty	DIR-008,017~018	
6	Coherence	Probe Hit/Miss	DIR-009	
7	Backpressure	请求/内存反压	DIR-010	

4 仿真环境

4.1 硬件配置

项目	规格
CPU	Apple M-series (ARM64)
内存	≥ 16GB
操作系统	macOS (Darwin)

4.2 软件环境

表 4: 软件环境

软件/工具	版本	说明
Python	3.11+	验证环境主语言
pytest	8.1.1	测试框架
pytest-cov	5.0.0	Python 覆盖率
pytest-html	4.1.1	测试报告生成
Picker	0.1.0-master-418d502	RTL→Python 导出
Verilator	5.046	RTL 仿真引擎
Java JRE	17.0.19	Chisel/NutShell 构建
Mill	0.11.7	Scala 构建工具
UCAgent	0.9.2+	AI 验证编排
Claude Code	0.131.0+	连接 UCAgent MCP Server, AI 辅助验证
WorkBuddy	latest	文档整理与报告生成
Makefile	—	workflow 启动管理 (回归/覆盖率/Bug 注入)
pytoffee / toffee-test	latest	Toffee 方法学与测试工具

4.3 安装与验证

Picker 安装流程：从源码克隆 → 创建虚拟环境 → 应用 Python 版本兼容性补丁 → `make init && make -j && make install`。安装完成后通过 Adder 示例进行 Smoke 验证 ($1+2=3$)。

NutShell Chisel 构建 (探索性)：通过 Mill 工具编译 Chisel 源码生成 Verilog RTL。构建命令为 `mill -i generator.test.runMain top.TopMain BOARD=sim CORE=inorder --split-verilog`。此构建为探索性质，最终 DUT 选定为 Picker example/Cache。

4.4 回归测试

回归测试通过 `scripts/run_regression.sh` 一键执行，当前状态：37 passed。覆盖率收集通过 `scripts/collect_coverage_multi.sh` 执行，支持多 Seed 聚合。可复现性通过 `scripts/reproduce.sh` 保证：[reproduce] PASS。

5 测试用例

5.1 Smoke Test（基本连通性测试）

验证 Cache 基本读写功能的正确性。测试流程：

1. 复位 DUT 并验证 io_empty 信号拉高；
2. 发送 Read Miss 请求，验证 Memory 请求产生；
3. 模拟 Memory Refill 响应（8 节拍），验证数据返回正确；
4. 再次发送相同地址 Read 请求（Hit），验证 ≤ 3 周期返回；
5. 发送 Write Hit 请求，写入后立即 Read 验证数据一致性。

测试结果：37 passed，全部通过。

5.2 Directed Tests（定向测试）

定向测试覆盖 27 个场景（DIR-001 至 DIR-022），针对 Cache 各功能路径进行精确验证：

表 5: 定向测试概览

编号	测试文件	验证目标
DIR-001/002	test_write_masks.py	部分字节写掩码验证
DIR-003	test_word_offsets.py	同 Cache Line 不同 Word Offset 读写
DIR-004	test_mmio_bypass.py	MMIO 旁路读写、Burst 拒绝、反压
DIR-005	test_refill_beats.py	8 节拍 Refill 完整性与非零偏移起始
DIR-006	test_invalid_way_replacement	无效路优先替换 vs 随机选择
DIR-007	test_dirty_writeback.py	脏替换 → 写回 → Refill 完整序列
DIR-008	test_flush_behavior.py	idle/in-flight flush 行为验证
DIR-009	test_coherence_probe.py	Coherence Probe Hit/Miss 响应
DIR-010	test_backpressure.py	请求/内存反压处理
DIR-011	test_clean_eviction.py	洁净替换不产生写回
DIR-012	test_write_miss_dirty_evict	写缺失 + 脏替换 + 部分掩码合并
DIR-013	test_write_miss.py	洁净写缺失 Refill + 写合并

所有定向测试均通过 `scripts/run_directed.sh` 执行，结果：27 passed。

5.3 Random Tests (约束随机测试)

约束随机测试 (CRV) 通过 `src/generator/cache_random.py` 生成确定性随机流量 (可复现 Seed)，覆盖以下随机维度：命令类型 (Read/Write/Probe 按权重分布)、地址范围 (覆盖命中和缺失场景)、写掩码类型 (全字节、单字节、多字节组合)。

随机测试通过 `scripts/collect_coverage.sh 7 18` 执行 (7 Seed × 18 步)，配合功能覆盖率收集。通过 Seed 多样性 (5→10→多 Seed) 和步数增加 (100→200→更多)，逐步提升覆盖率。

5.4 Bug 注入测试

Bug 注入测试用于证明验证环境的缺陷检出能力。共注入 6 个 Bug：

表 6: Bug 注入记录

Bug ID	注入方式	注入位置	检出机制	预期输出
BUG-001	Ref Model 数据 翻转	read_word() bit 0	Scoreboard.check_	Exit code 1 ()
BUG-RTL-001	RTL 状态机旁 路	Cache.v:615	test_dirty_writeback_indexError	
BUG-003	命中错误 Way	Tag 比较逻辑	Scoreboard 数据 比对	Exit code 1
BUG-004	替换 Way 错误	LFSR/无效路 选择	Scoreboard 替换 检查	Exit code 1
BUG-005	Refill 字节交换	节拍顺序	Scoreboard 多节 拍检查	Exit code 1
BUG-006	写回地址错位	Writeback Addr	Scoreboard 写回 序列检查	Exit code 1

所有 Bug 均被正确检出。正常回归测试保持 37 passed（通过 --disable-bug 恢复）。

6 覆盖报告

6.1 覆盖率总览

经过多轮覆盖率闭环迭代，当前覆盖率状态如下：

表 7: 覆盖率总览

覆盖率类型	已覆盖/总数	百分比	状态	备注
Line（行覆盖）	1,359 / 1,359	100.0%	达标	Category A-K 21 行豁免
Branch（分支覆盖）	471 / 471	100.0%	达标	Category L-N 23 分支豁免
Expr（表达式覆盖）	137 / 137	100.0%	达标	Category O 6 表达式豁免
Toggle（翻转覆盖）	24,947 / 28,227	88.4%	已豁免	T-A~T-F 3,280 项豁免
Toffee 功能覆盖	12 组 37 bins	100.0%	达标	31 覆盖点全通过

6.2 行覆盖率豁免分析

行覆盖率经过 Category A-K 豁免分析后达到 100%。主要豁免类别：

表 8: 行覆盖率豁免类别（部分）

类别	说明	豁免行数	豁免理由
A	断言 \$fwrite 失败消息	5 行	断言永不触发，仅用于错误报告
B	D-Cache forwarding 信号	4 行	I-Cache 配置下硬连线为 0
D	io_flush[1] 流水线 Kill	4 行	被 D-Cache 断言阻断，不可达
F	LFSR 种子初始化	2 行	全零为死状态（设计意图）
K	respToL1Last 计数器	3 行	I-Cache 单拍限制，不可达

6.3 分支覆盖率豁免

分支覆盖率在 Category L-N 豁免后达到 100%：

表 9: 分支覆盖率豁免类别

类别	说明	豁免数	豁免理由
L	CacheStage2 Forward-Meta MUX	10 分支	D-Cache forwarding 路径
M	CacheStage3 D-Cache Forwarding	5 分支	D-Cache forwarding 路径
N	DIR-019~022 目标分支	8 分支	I-Cache 结构上不可达

6.4 翻转覆盖率豁免

翻转覆盖率当前为 88.4% (24,947/28,227)，已豁免 3,280 项结构性无法翻转的信号 (Category T-A~T-F)：

表 10: 翻转覆盖率豁免类别

类别	说明	豁免理由
T-A	SRAM 地址/数据总线位	SRAM 行为模型宽度信号, 仅部分值出现
T-B	D-Cache 常量信号	I-Cache 配置下固定为 0/1
T-C	LFSR 替换位	伪随机 LFSR 需 2^{64} 周期覆盖全状态空间
T-D	断言专用条件信号	SVA 断言内部信号
T-E	复位后保持/固定信号	设计级常量或复位后不变
T-F	未使用/NC 端口位	IC 端口未连接位

经过多 Seed 策略验证 (5 Seed $\times$ 100 步 $\rightarrow$ 10 Seed $\times$ 200 步)，额外仿真未显著改善翻转覆盖，确认已达结构性平台期。

6.5 Toffee 分支覆盖率报告差异

在验证过程中发现 Toffee 报告流程存在数据差异：通过 `--toffee-report` 生成的 HTML 显示 C++ 级别 Branch 覆盖率为 85.0%，而 `code\_coverage.json` 中记录的 RTL 级别分支覆盖率为 95.3%（最终达 100%）。经分析确认：

- **Flow A** (LCOV genhtml)：统计 C++ 级别分支，Verilator 将 RTL 转换为 C++ 后产生大量额外分支。例如 1 行 wire 声明产生 128 个 C++ 分支，导致 28,949 个 C++ 分支与 494 个 RTL 分支的巨大差距；

- **Flow B** (`code_coverage.json`): 直接从 JSON 提取 RTL 级别覆盖率, 准确反映 DUT 的原始分支结构;
- 已生成修复脚本 `scripts/generate_rtl_coverage_html.py`, 直接展示 RTL 级别覆盖率。

7 总结

7.1 工作亮点

1. **完整的 AI 辅助验证工作流**: 从 GenSpec 规范生成、Stage 编排到覆盖率闭环, 全部通过 UCAgent 框架自动化执行, 累计完成 18 个 Stage;
2. **覆盖率全面达标**: 实现 100% 行覆盖、100% 分支覆盖、100% 表达式覆盖、88.4% 翻转覆盖 (已豁免) 和 100% Toffee 功能覆盖 (12 groups/31 points/37 bins);
3. **结构化 Toffee 验证环境**: Generator/Driver/Monitor/Scoreboard 四层分离, Scoreboard 从 35 行扩展到 194 行 (3 级 11 方法);
4. **缺陷检出能力证明**: 注入并正确检出 6 个 Bug, 覆盖参考模型注入和 RTL 注入两种方式;
5. **覆盖率报告修复**: 发现并修复 Toffee 覆盖率报告 Bug (C++ vs RTL 级别数据差异), 生成修复脚本确保报告准确性;
6. **完善的双语文档**: 覆盖验证全流程的中英文双语文档体系, 确保可复现性。

7.2 不足与改进

- **翻转覆盖率 88.4%**: 剩余 11.6% 的翻转覆盖缺口主要来自 SRAM 行为模型的宽位信号和 LFSR 穷举路径。虽已通过文档豁免 (Category T-A~T-F), 但从严格工程角度看, 部分信号可通过白盒测试覆盖;
- **Coherence 验证有限**: NutShell L1 Cache 不支持 Coherence 协议 (无 probe/release 状态机), 验证仅限于被动探针响应, 未覆盖完整一致性场景;
- **性能分析缺失**: 当前验证聚焦功能正确性, 未进行 Cache 性能分析 (命中率、延迟分布、带宽等);
- **仅覆盖 I-Cache 配置**: 由于 D-Cache 的 assert 阻断和 forwarding 路径复杂性, 验证限定在 I-Cache 单一配置。

7.3 未来展望

后续可扩展方向: 接入完整的 NutShell 多层次 Cache 系统 (L1 I-Cache + L1 D-Cache + L2 Cache) 进行集成级验证; 引入 Cache 一致性协议 (如 TileLink) 的完整验证; 性能分析 (Performance Analysis) 与覆盖率驱动的性能 Bug 检测; 将 UCAgent 工作流推广至其他 NutShell 组件 (如 TLB、流水线、分支预测器)。

8 结论

本报告完整记录了 NutShell Cache 模块的功能验证过程与成果。通过 UCAgent + Claude Code + WorkBuddy + Picker + Verilator + Toffee 技术栈，成功完成了从环境搭建到覆盖率闭环的全流程验证。

验证结果表明，NutShell L1 Cache 的 I-Cache 配置在功能层面表现正确：行覆盖、分支覆盖、表达式覆盖均达到 100%，翻转覆盖在豁免后可接受（88.4%），Toffee 功能覆盖全部 12 组 37 仓达到 100%。通过 6 个注入 Bug 的成功检出，验证了验证环境的缺陷检出能力。

在 AI 辅助验证方面，UCAgent 框架有效提升了验证效率：GenSpec 自动生成功能规范文档，Stage 编排自动化了验证迭代流程，覆盖率分析辅助了豁免决策。18 个 Stage 的完整执行记录为 AI 协同验证方法论提供了实证支持。

综上所述，NutShell Cache 的 I-Cache 功能验证已达到竞赛要求标准，验证环境具备可复现性，成果可用于后续集成验证和 AI 辅助验证方法学推广。